

SYNERGETICS • INSTRUCTOR-LED WORKSHOP

AI Model Orchestration & RAG

Day 2: RAG Patterns, Agentic RAG & Model Orchestration

RAG patterns • Agentic RAG • Multi-model orchestration

Learning Objectives

1

Apply RAG patterns

Use hybrid search, re-ranking, and multi-hop retrieval to sharpen retrieval quality.

2

Understand agentic RAG

See how retrieval combines with autonomous agent reasoning to handle harder questions.

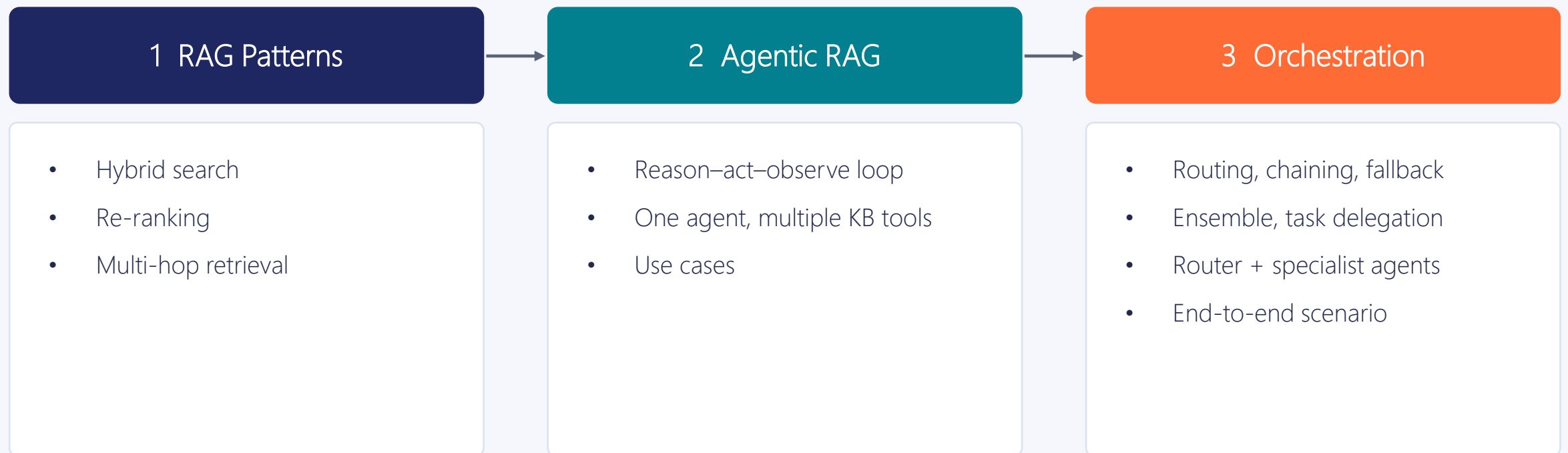
3

Orchestrate across models

Apply routing, chaining, fallback, ensemble, and task delegation, and choose the right one for the job.

Day 2 Overview

Today builds up in three steps: sharper retrieval first, then agents that decide how to retrieve, then patterns for coordinating several models.



RAG Patterns: Hybrid Search

CONCEPT

Vector search alone can miss exact matches (product codes, error IDs, acronyms). Hybrid search combines vector similarity with traditional keyword search in a single query.

Why it matters

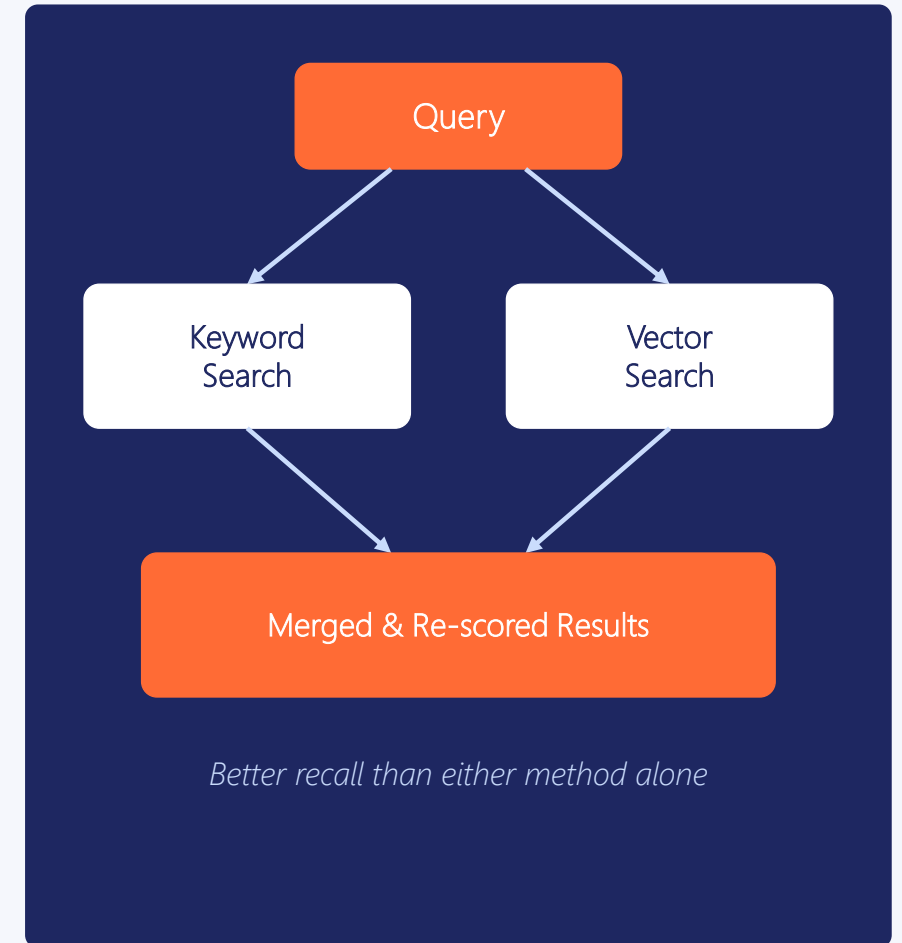
Keyword search catches exact terms; vector search catches meaning. Together they cover more real queries.

How it works

The query runs as both a keyword search and a vector search; results are merged and scored together.

Azure AI Search

Supports hybrid search natively — both kinds of query run together in a single request.



Hybrid Search: A General Example

EXAMPLE

Question: "My printer shows error E4021. How do I fix it?"

Keyword search

Fix printer error E4021

Error code reference: E4021

Missed: "Printer stops after a paper jam" (no code in the text)

Exact words, but no meaning

Vector search

Printer stops after a paper jam

Troubleshooting printer errors

Risk: may return pages for similar codes like E4012 instead of E4021

Meaning, but blurry on codes

Hybrid search (merged)

Fix printer error E4021

Printer stops after a paper jam

Troubleshooting printer errors

Exact match and meaning

Everyday picture: the index card finds the exact title, the librarian finds books on the same topic. Hybrid uses both.

RAG Patterns: Re-Ranking

CONCEPT

Initial retrieval (hybrid or vector) is optimized for speed across a huge index. Re-ranking applies a more accurate model to just the top candidates to reorder them by true relevance.



Why it matters: the top result from the first pass is not always the most relevant one. Re-ranking recovers the precision that a fast, approximate first pass gives up for speed. In Azure AI Search, the semantic ranker provides this as a single option.

Re-Ranking: A General Example

EXAMPLE

Question: "How many days of leave do new employees get?"

Before: first search (fast, rough)

1 General leave overview

2 Holiday calendar

3 Leave request process

7 New employee leave entitlement

The right answer is buried at position 7

Re-ranker
reads the question and each
result, then scores relevance

After: re-ranked (careful)

1 New employee leave entitlement

2 General leave overview

3 Leave request process

Only the top few results go to the model

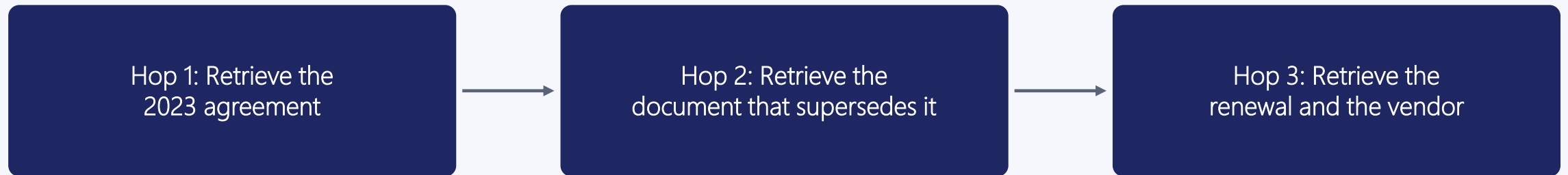
Everyday picture: hiring. Scan 50 resumes quickly, then interview the shortlist and rank it properly.

RAG Patterns: Multi-Hop Retrieval

CONCEPT

Some questions can't be answered from a single retrieved chunk — the answer requires connecting information across several documents, retrieved in sequence.

Example: "Which vendor renewed the contract that superseded our 2023 data-processing agreement?"



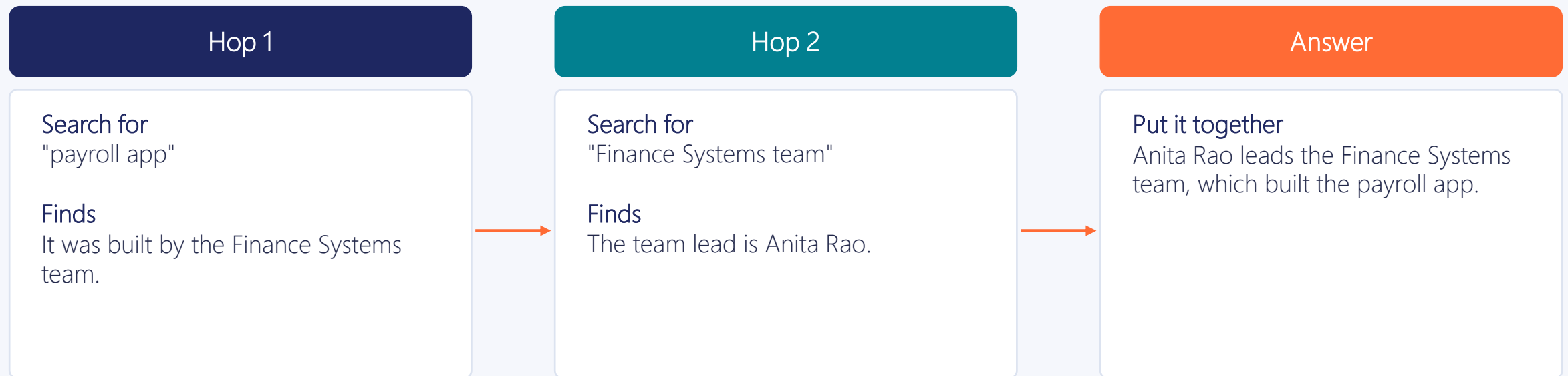
Each hop's result informs the next retrieval query — this is where RAG starts to look like reasoning, not just lookup.

Multi-Hop Retrieval: A General Example

EXAMPLE

Question: "Who leads the team that built the payroll app?"

No single document contains both facts, so one search is not enough. Each hop uses what the previous hop found.



Everyday picture: a detective. One clue leads to a person, and that person leads to the next clue.

RAG Patterns: Examples & Use Cases

EXAMPLES

1 Technical Support

Hybrid search catches exact error codes that pure vector search would blur into generic "similar" results.

2 Legal Discovery

Re-ranking surfaces the single most relevant clause among hundreds of superficially similar contract passages.

3 Financial Research

Multi-hop retrieval connects a company's filing to a related subsidiary's filing to answer a compound question.

4 Regulatory Compliance

Combining all three: hybrid search finds candidates, re-ranking prioritizes them, multi-hop chases cross-references.

Agentic RAG: What & Why

CONCEPT

In standard RAG, retrieval happens once, in a fixed spot in the pipeline. In agentic RAG, an agent decides whether, when, and what to retrieve — potentially several times — as part of its own reasoning.

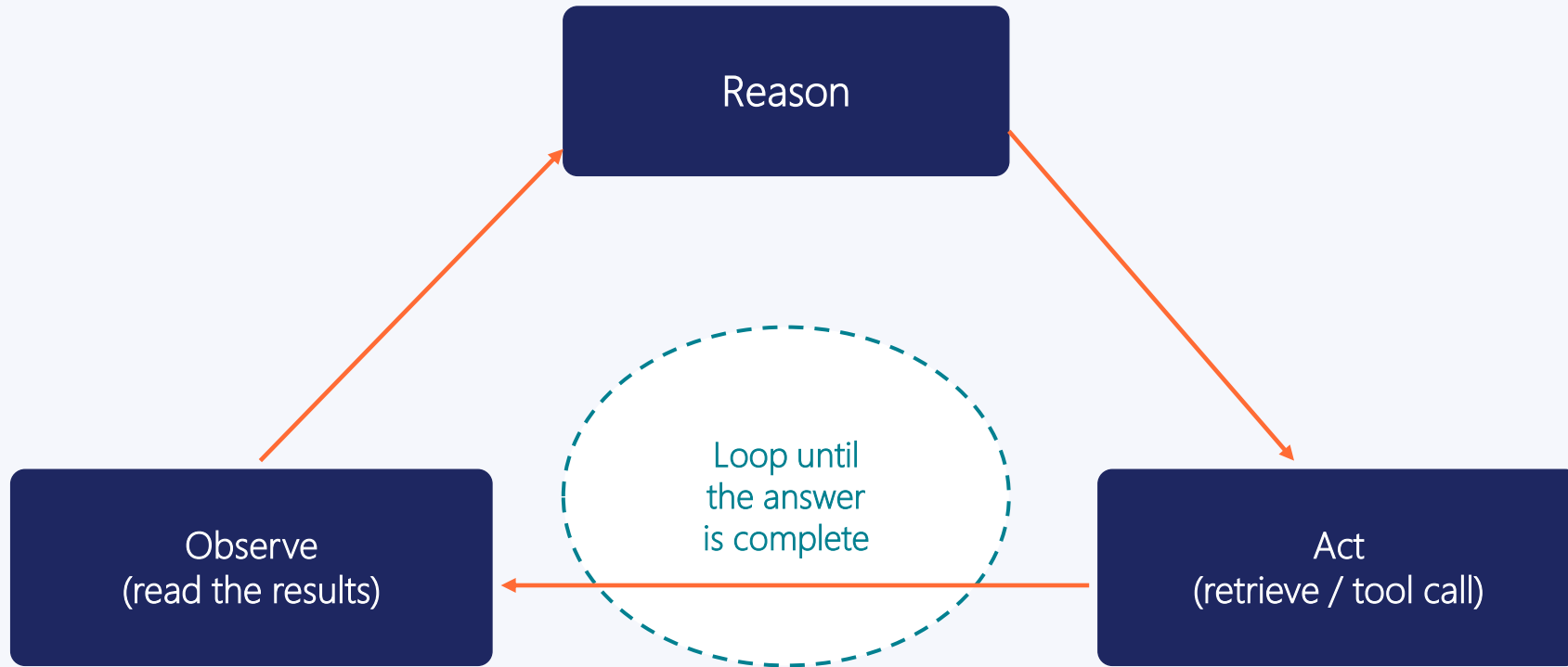
Standard RAG

- Fixed pipeline: retrieve once, then generate
- Good for direct, single-source questions
- Predictable cost and latency

Agentic RAG

- Agent decides if, when, and what to retrieve — possibly repeatedly
- Good for multi-step, ambiguous, or multi-hop questions
- Variable cost and latency — more capable, less predictable

How Agentic RAG Works: Reason–Act–Observe

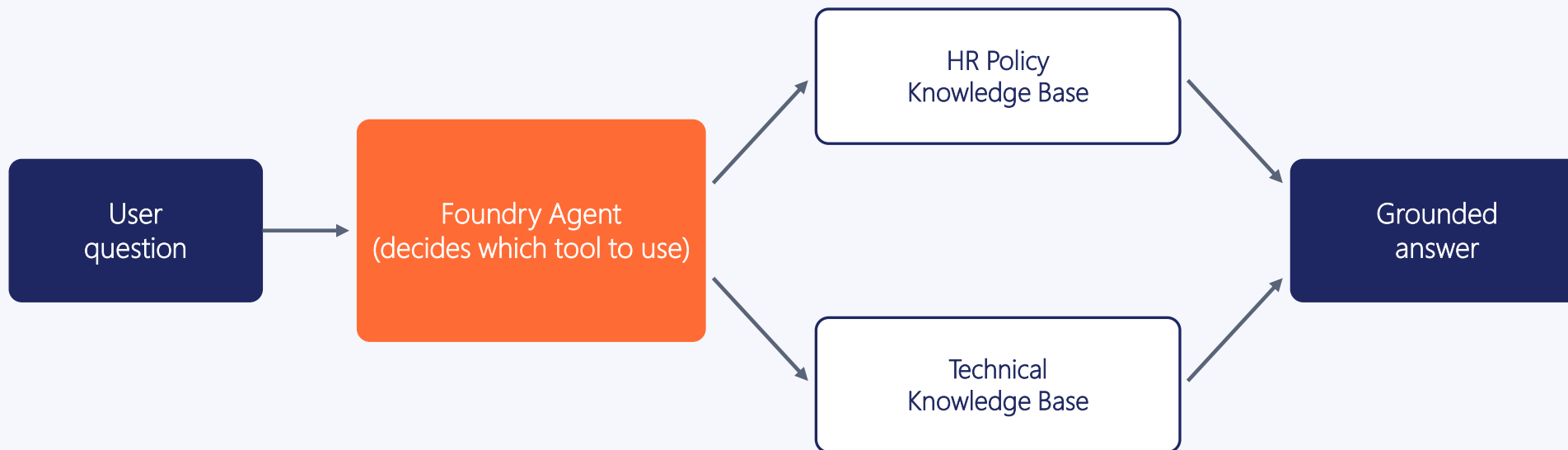


The agent reasons about what it needs, acts (often a retrieval call, sometimes another tool), observes the result, and decides whether to loop again or answer. This is the mechanism behind multi-hop retrieval.

Agentic RAG in Foundry: One Agent, Multiple KBs

PATTERN

The basic, tool-based form of agentic RAG: the agent itself decides which knowledge source to consult.



Strength

Simple, few parts

Watch out

Less predictable as sources grow

Both knowledge bases are attached as tools to one agent — this is not a multi-agent hand-off.

Agentic RAG in Practice: Use Cases

EXAMPLES

1 Deep Research Assistant

Iteratively searches, reads, and refines its own follow-up queries until it has a complete answer.

2 Cross-Document Investigation

Chases references across contracts, filings, or tickets without a human specifying every step.

3 Clarify-Then-Answer Support Bot

Recognizes an ambiguous question and asks a clarifying question before retrieving anything.

4 Self-Checking Analyst

Retrieves an answer, then retrieves again to verify it before presenting it to the user.

The Five Orchestration Patterns

CONCEPT

Production systems combine several patterns. Routing, chaining, and fallback are the core building blocks; ensemble and task delegation are used selectively.

CORE BUILDING BLOCKS

Routing

Pick the right path or model for each request.

Chaining

Run a fixed sequence of steps, each feeding the next.

Fallback

If a step fails, automatically retry with a backup.

USED SELECTIVELY

Ensemble

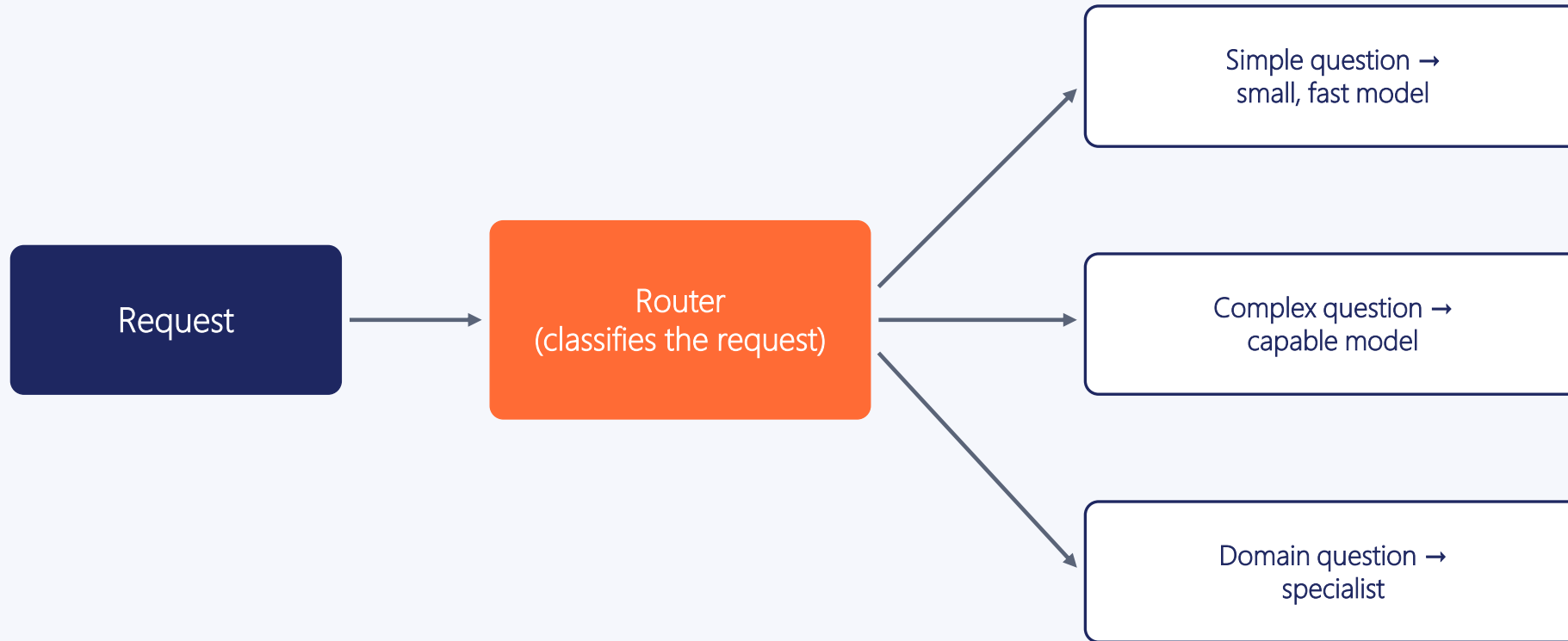
Several models answer; combine or vote on the result.

Task Delegation

A coordinator splits a goal and assigns sub-tasks to specialists.

Pattern: Routing

CONCEPT

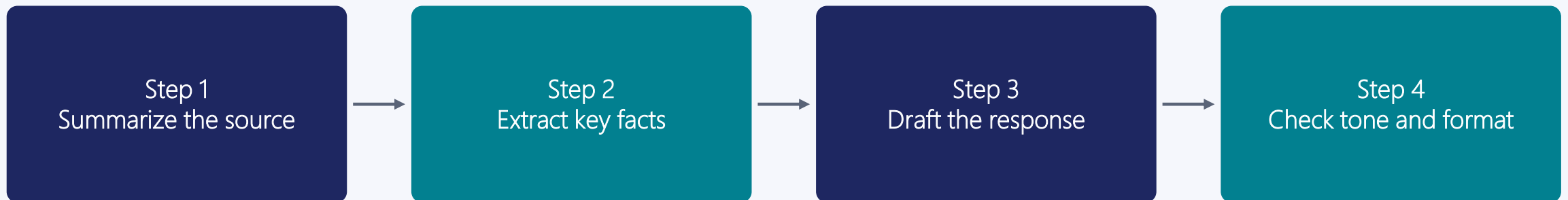


Use it for: requests that differ in kind and need different handling. Only one path runs per request, so cost stays low.

Pattern: Chaining

CONCEPT

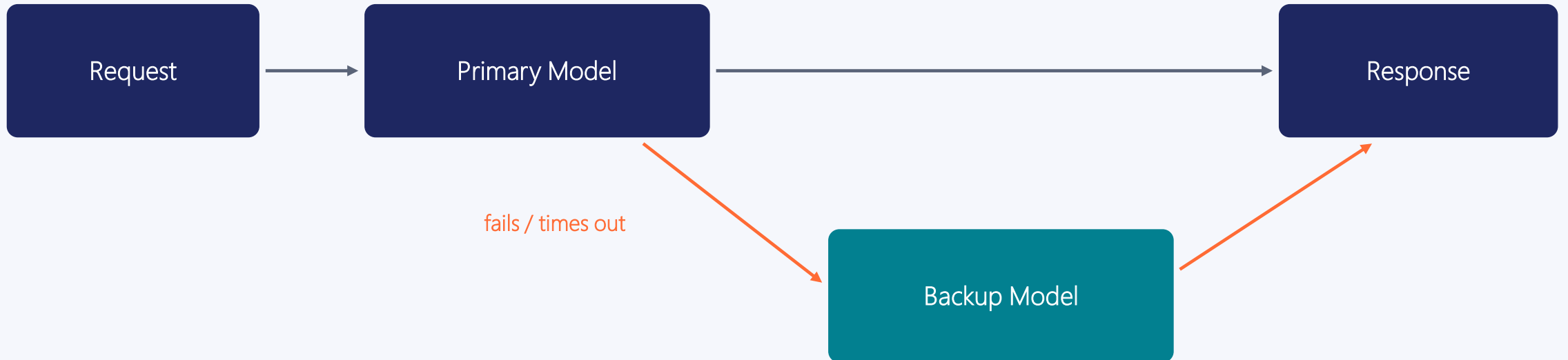
Each step's output becomes the next step's input — the order never changes.



Use it for: requests that break into a fixed, known sequence of steps. Trade-off: every step runs on every request, so cost and latency add up.

Pattern: Fallback

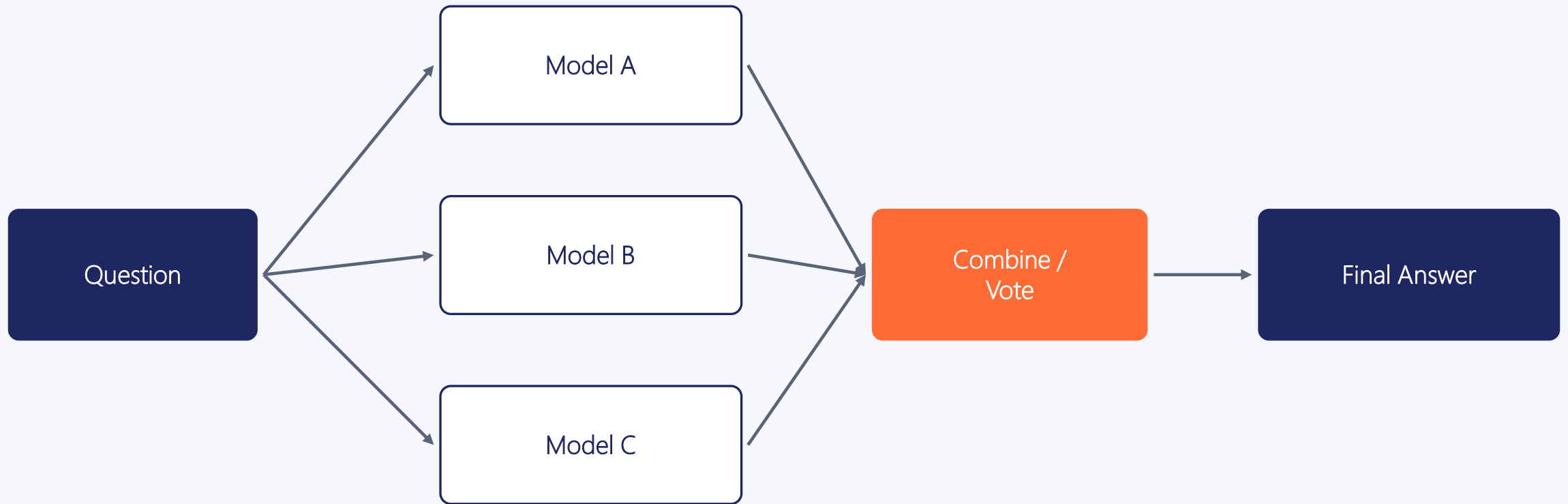
CONCEPT



Use it for: reliability against outages, rate limits, or regional model unavailability. The backup runs only on failure, so normal cost stays low.

Pattern: Ensemble

CONCEPT



Use it for: high-stakes decisions where one model's error is costly. Trade-off: several model calls on every request.

Pattern: Task Delegation

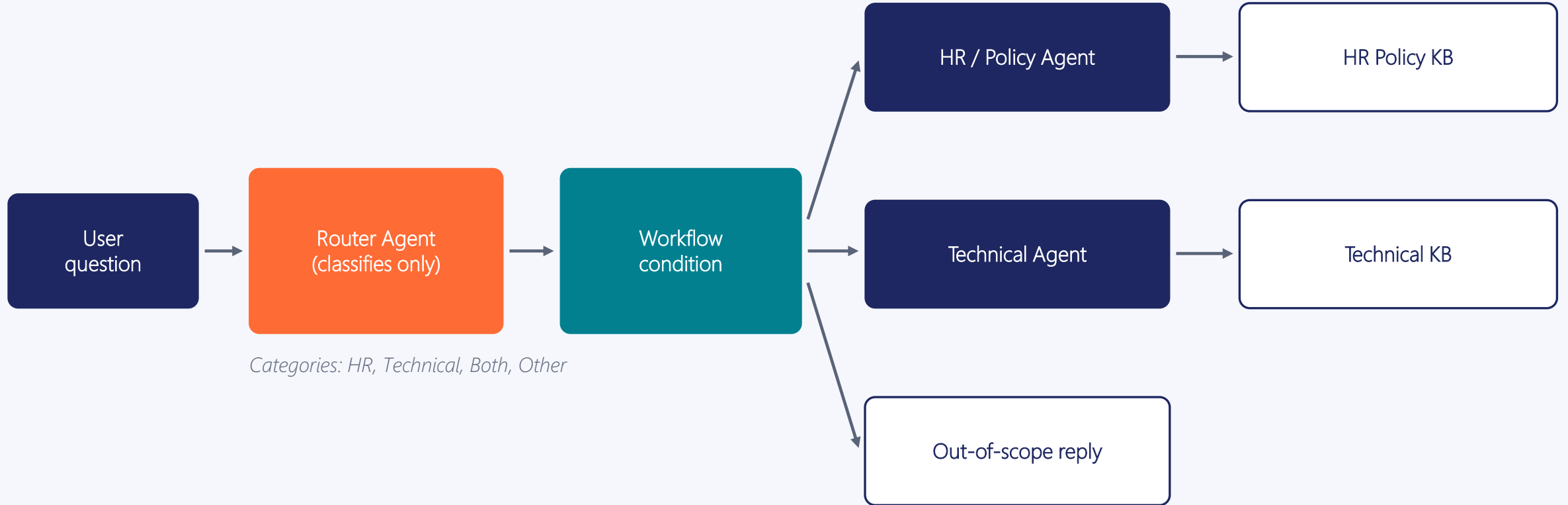
CONCEPT



Use it for: open-ended goals that break down differently each time. Unlike chaining, the coordinator decides which specialists are needed and in what order, based on the request.

Foundry Pattern: Router + Specialist Agents

PATTERN



Key point: creating several agents does not make one call another — the workflow performs the hand-off. The "Both" category needs extra logic to combine two answers.

One Agent with Tools vs Router + Specialists

COMPARISON

	One agent with multiple KB tools	Router + specialist agents
Who decides the path	The agent, from its instructions and tool descriptions	The workflow, from the router's category
Setup effort	Low — few components	Higher — more agents and workflow steps
Predictability	Can vary as sources grow	Explicit, inspectable routing decisions
Best when	Few, easy-to-describe sources; first working RAG	Distinct domains that need specialist behavior and control
Watch out for	Hard to see why a tool was chosen	Router mistakes send questions to the wrong branch

Orchestration Patterns: Which One, When?

Pattern	Use when...	Cost / complexity
Routing	Requests differ in kind and need different handling	Low — one path chosen per request
Chaining	A request breaks into a fixed, known sequence of steps	Moderate — every step runs every time
Fallback	Reliability against failures, rate limits, or outages matters	Low — backup runs only on failure
Ensemble	A wrong answer is costly and worth paying for consensus	High — several model calls every time
Task Delegation	Goals are open-ended and break down differently each time	High — dynamic planning plus specialist calls

Scenario: Enterprise Policy & Technical Assistant

SCENARIO

An employee assistant answers questions across HR policy documents and technical documentation. Follow one question through the system:



Chaining ties the stages together in a fixed order. Combine patterns deliberately — use each one only where it earns its cost.

Key Takeaways & Wrap-Up

1

RAG patterns sharpen retrieval

Hybrid search, re-ranking, and multi-hop each fix a different weakness.

2

Agentic RAG adds reasoning

An agent decides if, when, and what to retrieve — more capable, less predictable.

3

Five orchestration patterns, one toolkit

Routing, chaining, fallback, ensemble, and task delegation — combined as needed, not used everywhere.

Next steps & learning resources

- [Azure AI Search documentation](#) — hybrid search and semantic ranker
- [Azure AI Foundry documentation](#) — agents and workflows
- Try adding one orchestration pattern to your own pipeline and see what changes